

Grab'N'Go: Project Architecture & Code Review Document

1. Project Overview & Site Flow

Grab'N'Go is a comprehensive multi-vendor rental platform where users can rent items (electronics, furniture, vehicles, etc.) from various shop owners. The system supports three main roles: **Admin**, **Shop Owner**, and **Customer**.

End-to-End User Flow:

- Landing & Onboarding:** Users arrive at the Landing page (/). They can choose to Register (/register) as a Customer or a Shop Owner. Registration requires email verification via OTP (/otp).
- Authentication:** Users log in (/login). The backend issues a JWT (JSON Web Token) which the frontend stores in localStorage. The AuthContext uses this token to keep the user persistently logged in.
- Role-Based Routing:**
 - Customers** are taken to /dashboard to browse categories, view items, and make bookings.
 - Shop Owners** are taken to /shop-owner/dashboard to manage their inventory and view their shop's rentals.
 - Admins** are taken to /admin/dashboard to oversee the platform, approve new shops, manage categories, and view revenue analytics.
- Booking & Payment Checkout:** Customers browse items, select dates, and proceed to booking (/booking/:shopItemId). Real payments are simulated/handled using the Razorpay integration.
- Post-Booking:** Customers can view their active rentals (/active-rentals) and leave reviews (/reviews) once a rental is completed.

2. Database Schema & Tables

The database is built on **PostgreSQL**. The exact structure is defined in SQL architecture, but conceptually, it uses the following primary tables and relationships:

- users:** Stores all accounts. Fields: id, fullname, email, password (hashed), role (admin/customer/shop_owner), verified (boolean for OTP), blocked.
- shops:** Represents a vendor's store. Links to users.id via owner_id. Fields: name, address, is_active, rating.
- categories:** Global categories managed by Admins (e.g., Electronics, Vehicles).
- items:** Global master catalog of items linked to categories.
- shop_items:** The crucial mapping table! A specific shop offering a specific global item. Contains shop-specific pricing (price_per_day_inr), stock (quantity_available), and specific rules.
- bookings / rentals:** Tracks when a user rents a shop_item. Fields: start_date, end_date, total_amount, status (pending, active, completed, cancelled).
- payments:** Logs transactions linked to a booking_id. Contains Razorpay order_id and payment_id.
- reviews:** Ratings and text feedback left by users on shop_items, updating the average rating on both the item and the shop.

3. Backend Architecture (Node.js + Express)

The backend is structured in a modular fashion inside backend/src/.

Directory Structure

- config/db.js:** Establishes the connection pool to PostgreSQL using the pg library.
- middlewares/:**
 - authMiddleware.js:** Intercepts incoming requests, reads the Authorization: Bearer <token> header, verifies the JWT signature, and attaches req.user so protected routes know who is making the request.
 - upload.js:** Configures multer to handle multipart/form-data for image uploads (saving to uploads/ directory).
- modules/:** The core business logic is grouped by feature (e.g., /users, /bookings, /items). Every module generally has:
 - .routes.js:** Defines the API endpoints (e.g., router.post('/login', authController.login)).
 - .controller.js:** The brains of the operation. Handles the HTTP Request and Response, calls database queries, and returns JSON.

Code Highlight: server.js Flow

```
// 1. App Initialization
const express = require('express');
const app = express();

// 2. Global Middlewares
app.use(cors()); // Allows frontend running on port 5174 to talk to backend on 5000
app.use(express.json()); // Parses incoming JSON payloads into req.body

// 3. Mount Route Modules
const userRoutes = require('./modules/users/user.routes');
// ... other routes ...
app.use('/api', userRoutes);

// 4. Start Server
app.listen(PORT, () => console.log(`Server running...`));
```

4. Frontend Architecture (React + Vite + Tailwind)

The frontend is a Single Page Application (SPA) built with React and heavily styled with Tailwind CSS responsive classes (enabling our Light/Dark theme!).

Key Concepts

- React Router Dom (App.jsx):** We define routes here. We use custom wrapper components like <ProtectedRoute> that check if the user is logged in. If not, it forces them back to /login.

- **Context API:**
 - `AuthContext.jsx`: A secure vault that wraps the entire app. It holds the `user` object and provides `login/logout/register` functions to any component that needs them using `useAuth()`.
 - `ThemeContext.jsx`: Reads `LocalStorage` and user's system preferences to dynamically attach `class="dark"` to the HTML root tag, instantly flipping Tailwind styles.
- **Services:** In `frontend/src/services/`, files like `api.js` configure an `axios` instance that automatically injects the JWT token into every outgoing API request.
- **Tailwind CSS:** Instead of standard CSS files, we use utility classes. For example, `bg-white dark:bg-[#111827]` tells the browser: "Be white normally, but be dark navy if the parent has the `.dark` class."

Code Highlight: Frontend API Request (`item.service.js`)

When the `CustomerDashboard` loads, it needs data. It calls a `service` file:

```
// Example of how the frontend talks to the backend
import api from './api'; // Our pre-configured axios agent

const itemService = {
  getAllItems: async () => {
    // Sends a GET request to http://localhost:5000/api/items
    const response = await api.get('/items');
    return response.data; // Returns the JSON array to the React component
  }
}
```

Code Highlight: Component Rendering (`CustomerDashboard.jsx`)

```
// A React component receives data and renders JSX (HTML inside Javascript)
const CustomerDashboard = () => {
  // 1. Define state variables to hold our data
  const [items, setItems] = useState([]);

  // 2. useEffect runs automatically when the page loads
  useEffect(() => {
    // Fetch data from backend and update state
    itemService.getAllItems().then(data => setItems(data));
  }, []);

  // 3. Render the UI
  return (
    <div className="bg-gray-50 dark:bg-[#0B0F19]">
      {/* Loop through items and render an ItemCard for each */}
      {items.map(item => (
        <ItemCard key={item.id} data={item} />
      ))}
    </div>
  );
};
```

Summary of Information Flow

1. **User Action:** The user clicks "Rent Now" on the frontend UI (`BookingPage.jsx`).
2. **Frontend Service:** React tells `booking.service.js` to shoot an HTTP POST request to `/api/bookings` with the item details and picked dates.
3. **Backend Middleware:** `authMiddleware.js` verifies the user's JWT token to ensure they are a real logged-in customer.
4. **Backend Controller:** `bookingController.js` calculates the total price, verifies inventory space in the `shop_items` table, and runs an SQL `INSERT` statement into the `bookings` table.
5. **Database:** PostgreSQL rigidly saves the transaction and returns a success ID.
6. **Response:** Express sends a 200 OK JSON response back to the frontend.
7. **UI Update:** React sees the success, triggers a `toast.success()` notification, and redirects the user to `/active-rentals`.